

Posts and Telecommunications Institute of Technology



ASSIGNMENT 1

INTELLIGENT SYSTEM DEVELOPMENT

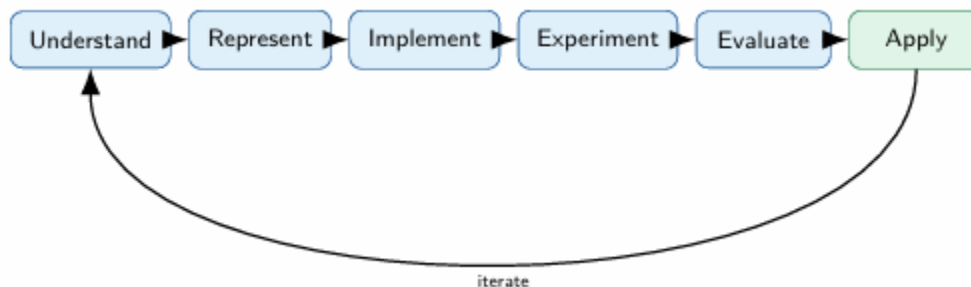
Lecturer	: TRẦN ĐÌNH QUẾ
Student	: MAI ANH TUẤN ANH
Student ID	: B23DCCE005
Class	: E23CNPM02

Hà Nội, August 2026

1. Introduction

- The purpose of this assignment is not simply to train a machine learning model, but to develop and explain a small intelligent system from a real-world problem.

This assignment is a progression from understanding intelligence to building systems that can represent information, learn from data, make decisions, act, and adapt. The development process is summarized as:



- Besides, an intelligent system is more than a trained machine learning model. A complete system connects data and computational methods to an environment through representation, interfaces, interaction, evaluation, deployment, and feedback.

For this assignment, the course starts with a relatively simple setting:

Structured data → Feature vector → Traditional ML → Application

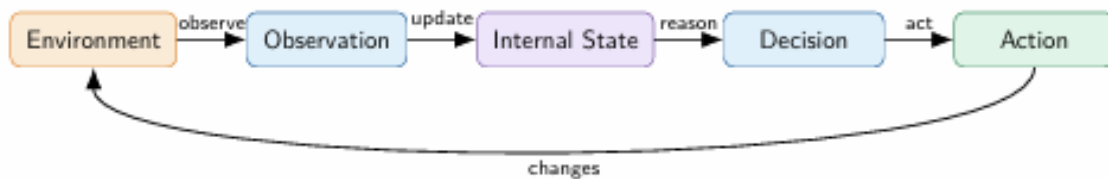
2. Intelligent System Definition

- An intelligent system can be understood as a computational system that interacts with an environment, processes information through an internal representation, learns or reasons from that information, makes decisions, and produces actions or outputs. In this view, intelligence is not associated with a single algorithm. It emerges from the coordination of several capabilities, including perception, representation, learning, reasoning, decision making, and action.

- A useful abstraction of an intelligent system is:

Intel Syst =

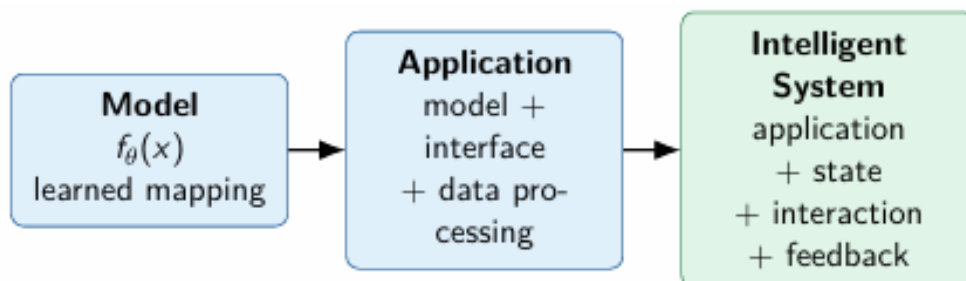
(Environment, State, Representation, Knowledge, Learning, Decision, Action)



- The environment contains the external information with which the system interacts. The system does not have direct access to the complete state of the environment. Instead, it receives observations and constructs an internal representation of the information that is currently available. This distinction between external state and internal state is important because the information available to the learning process depends on how the environment is represented.

Representation therefore plays a central role in intelligent system development. A learning algorithm does not operate directly on a real-world object, such as a customer, image, or document. It operates on a computational representation of selected information. Different representations preserve different information and consequently support different learning mechanisms.

An intelligent system should also be distinguished from a machine learning model or an application. A model is a learned function that maps an input representation to an output. An application combines the model with data processing and an interface so that the model can be used in practice. An intelligent system represents a broader structure in which the application operates within an environment and may incorporate state, interaction, knowledge, and feedback.



Therefore, the development of an intelligent system is not limited to model training. It involves a complete process of understanding the problem, determining

what information is available, selecting an appropriate representation, implementing a learning mechanism, conducting experiments, evaluating the results, deploying the resulting system, and iterating based on evidence.

3. Problem Definition

- The problem addressed in this project is a binary classification task for diabetes prediction.

For each patient, the system receives eight medical measurements and predicts whether the patient belongs to the diabetes-positive class.

The target variable is Outcome:

0 → Negative class

1 → Positive class

For each observation, the system has an input representation x and a target y . The learning problem can be expressed as:

$$D = \{(x_i, y_i)\}$$

where x_i is the input feature representation and y_i is the corresponding target.

The model learns a function:

$$\hat{y} = f_{\theta}x$$

where θ represents the learned parameters of the model and $\hat{y}$ is the predicted class.

Given:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N,$$

we seek:

$$\hat{y} = f_{\theta}(x).$$

For classification:

$$\hat{y} \in \{0, 1\}.$$

The learning process searches for:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i).$$

- The data is divided into a training set and a test set. The training set is used for learning, while the test set is kept unseen during model development and is used to estimate performance on new observations.

4. Dataset

- The dataset used in this project is diabetes.csv. It contains 768 observations and 9 columns, consisting of 8 input features and one target variable.

```
Dataset loaded successfully.  
Shape: (768, 9)
```

```
Feature columns:  
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']
```

```
Target:  
Outcome
```

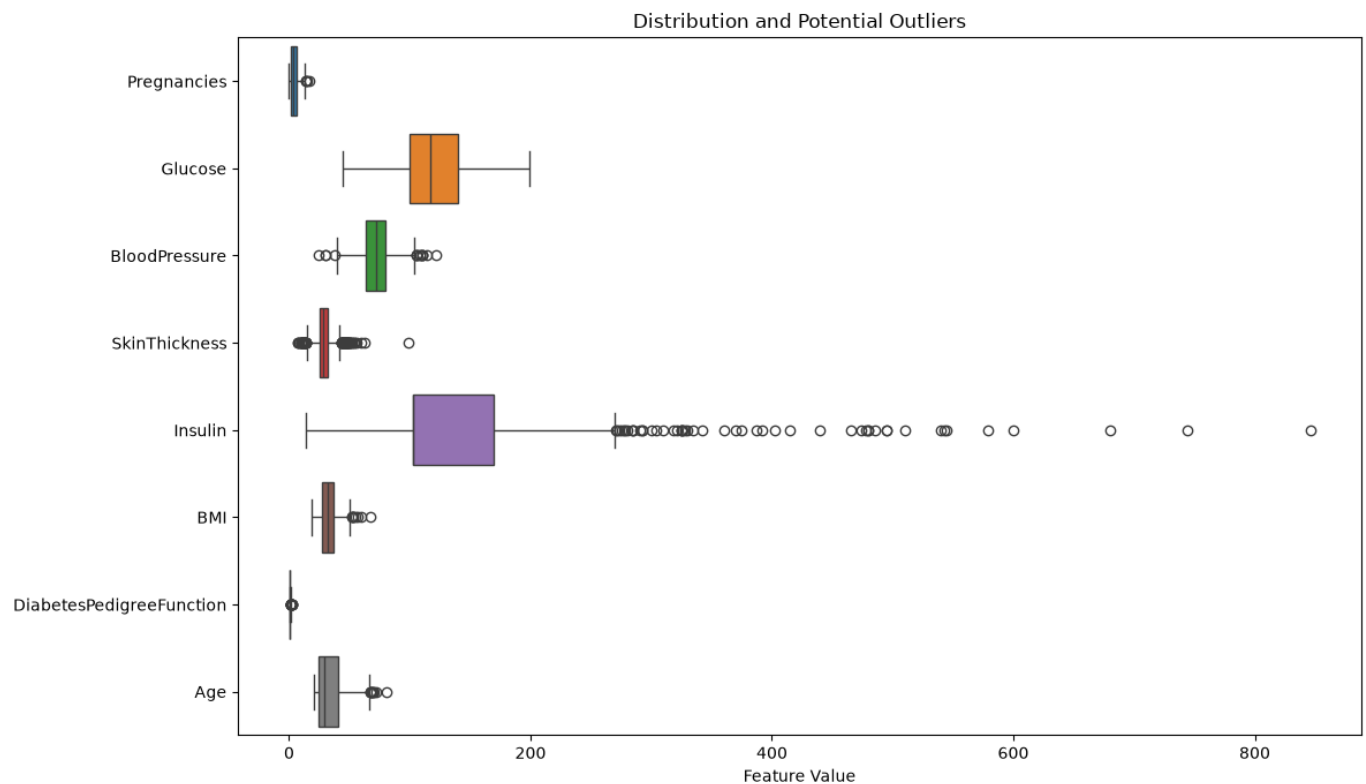
```
Input shape: (768, 8)  
Target shape: (768,)
```

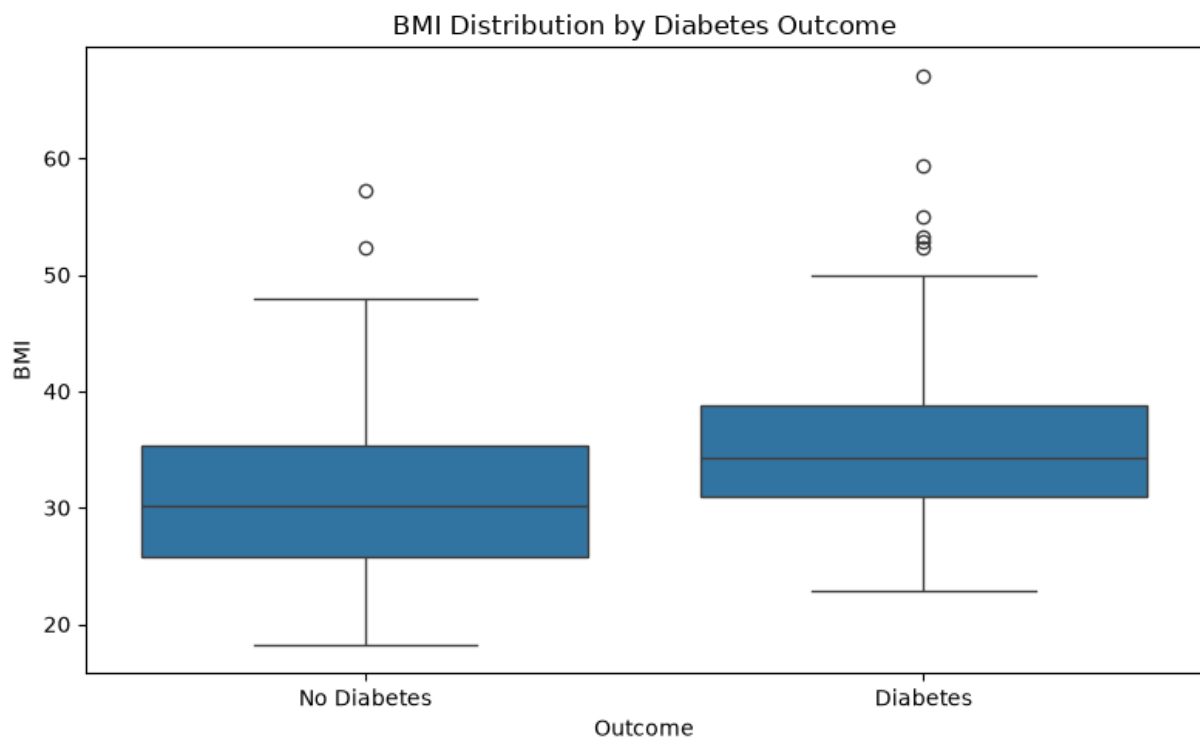
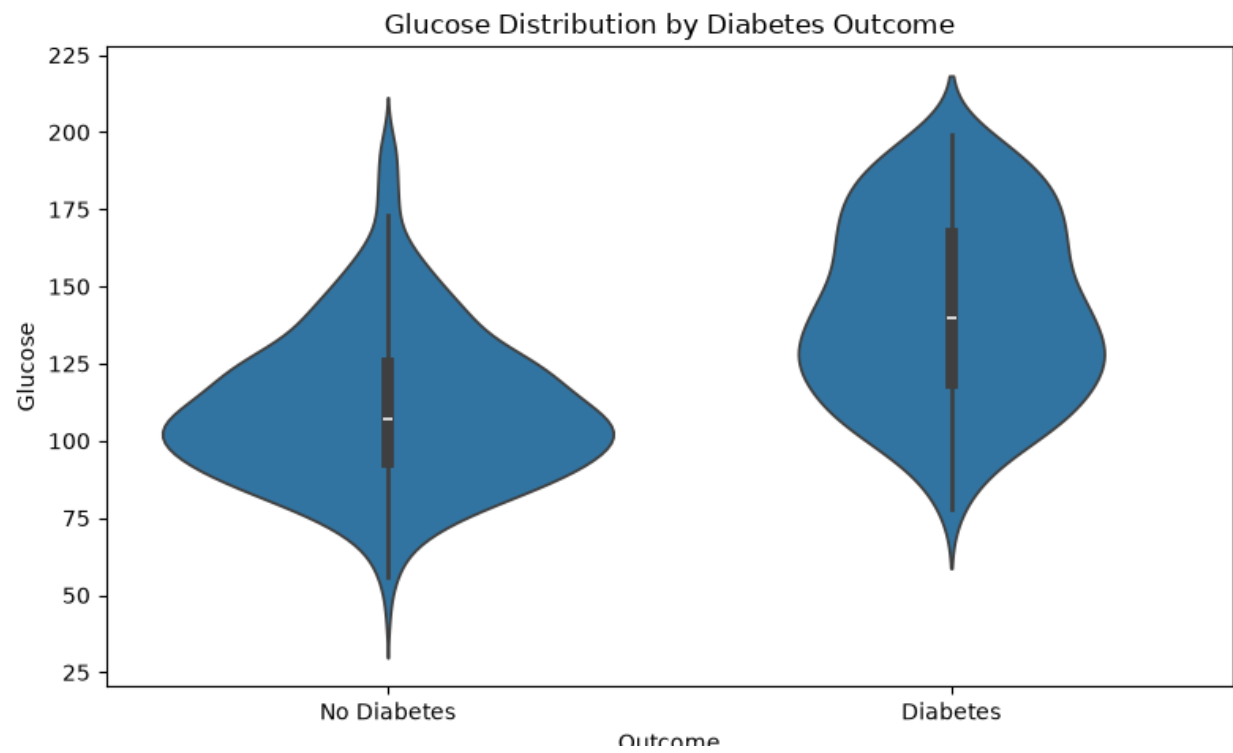
Here is its source: [Kaggle: Your Home for Data Science](https://www.kaggle.com/uciml/diabetes)

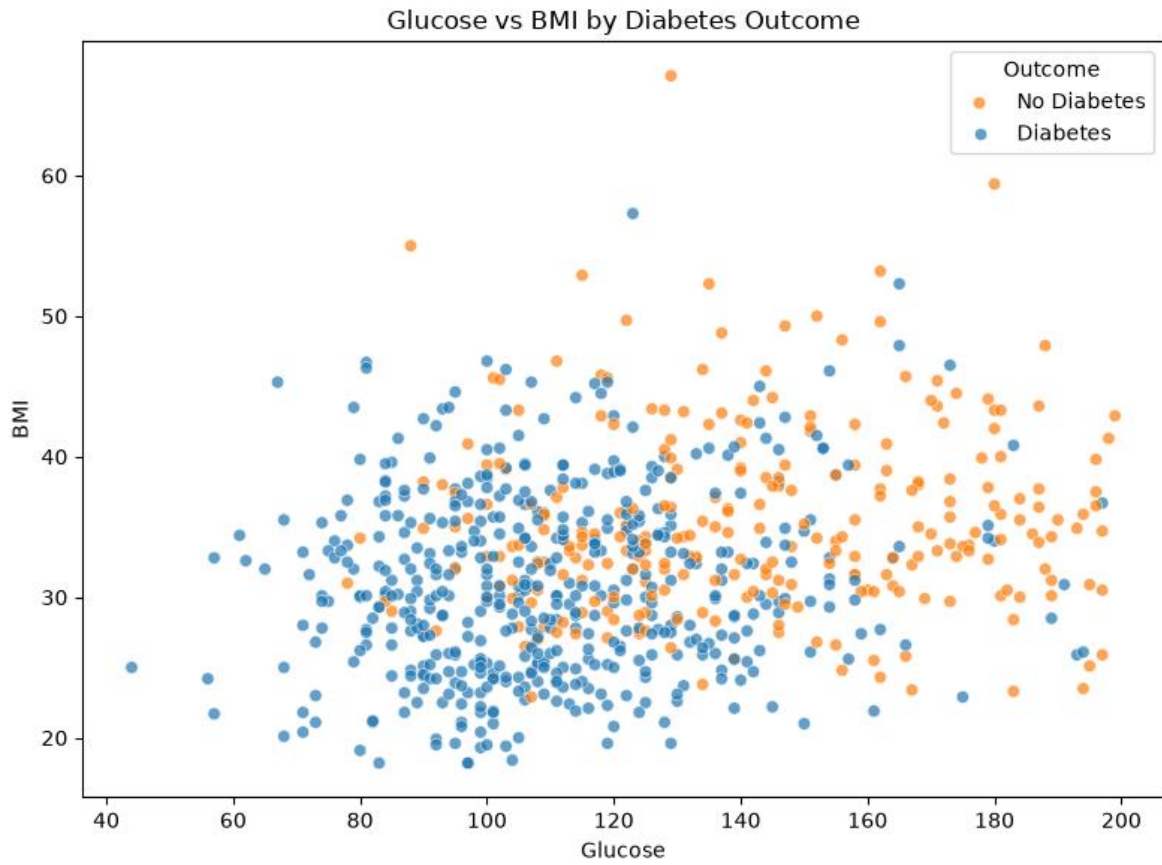
All input variables are numerical. Consequently, the project does not require categorical encoding such as One-Hot Encoding.

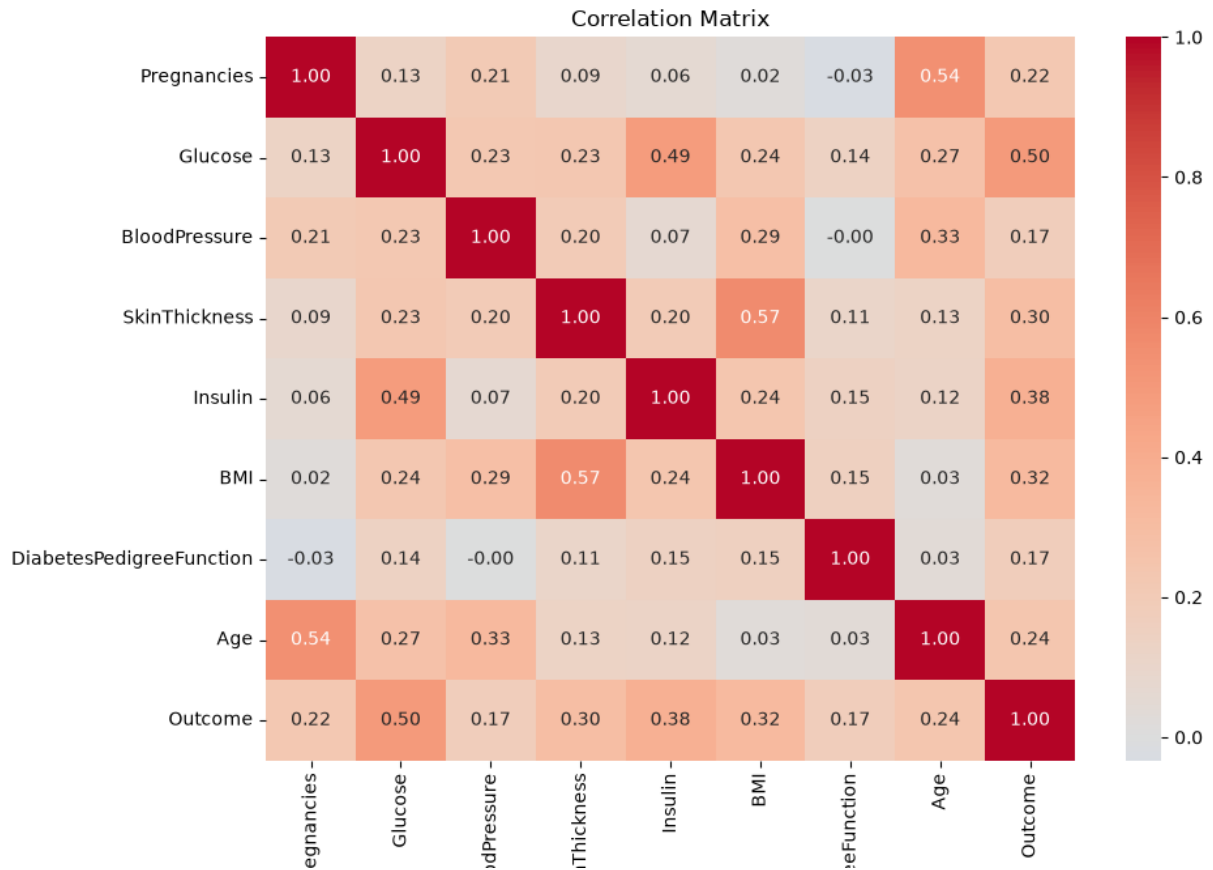
Feature	Type	Meaning
Pregnancies	Numerical	Number of pregnancies
Glucose	Numerical	Plasma glucose concentration
BloodPressure	Numerical	Diastolic blood pressure
SkinThickness	Numerical	Triceps skin-fold thickness
Insulin	Numerical	Serum insulin level
BMI	Numerical	Body Mass Index
DiabetesPedigreeFunction	Numerical	Diabetes pedigree function score
Age	Numerical	Age of the patient
Outcome	Target	Diabetes classification label

- There are some data distribution diagram:









5. Data Representation

- The machine learning algorithm does not receive the real-world client directly. So that, the client is converted into a computational representation. For structured data, this representation is a feature vector:

$$x = [x_1, x_2, \dots, x_d]$$

The preprocessing pipeline is:

Raw patient measurements



Selected numerical features



Median imputation



Standardization



Numerical feature vector



ML model

Median imputation is used to provide a robust value when a numerical measurement is missing.

Standardization transforms numerical variables into a common scale. This is particularly important for KNN because KNN uses distances between observations.

The standardized feature vector is then provided to the traditional machine learning models.

Zero-value counts in selected medical measurements:

Zero Count	
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0

	Feature	Type	Representation
0	Pregnancies	Numerical	Imputed if necessary + Standardized
1	Glucose	Numerical	Imputed if necessary + Standardized
2	BloodPressure	Numerical	Imputed if necessary + Standardized
3	SkinThickness	Numerical	Imputed if necessary + Standardized
4	Insulin	Numerical	Imputed if necessary + Standardized
5	BMI	Numerical	Imputed if necessary + Standardized
6	DiabetesPedigreeFunction	Numerical	Imputed if necessary + Standardized
7	Age	Numerical	Imputed if necessary + Standardized

6. Traditional ML Methods

- Four traditional machine learning models were trained:

+ Logistic Regression

+ K-Nearest Neighbors

+ Decision Tree

+ Random Forest

The purpose of using several models is not simply to find the model with the highest score. The four models use the same basic feature representation but apply different learning mechanisms and assumptions. This is an important characteristic of traditional machine learning: different algorithms can receive essentially the same representation while learning different relationships from it. We will use 5-fold cross-validation protocol to train each model.

- Logistic Regression

Logistic Regression learns a weighted relationship between the feature vector and the probability of the positive class.

For binary classification, the model computes:

$$z = w^T x + b, \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

where σ is the sigmoid function.

Each feature contributes through a learned weight. A decision threshold is then used to convert the probability-like score into a class prediction.

In this project, the model receives the preprocessed feature vector containing standardized numerical features and one-hot encoded categorical features.

The learned parameters are the feature weights and the intercept.

```
from sklearn.linear_model import LogisticRegression

model_1 = Pipeline([
    (
        "preprocess",
        preprocessor
    ),
    (
        "model",
        LogisticRegression(
            max_iter=1000,
            random_state=RANDOM_STATE
        )
    )
])

model_1_scores = cross_validate(
    model_1,
    X_train,
    y_train,
    cv=cv,
    scoring=[
        "accuracy",
        "precision",
        "recall",
        "f1"
    ],
    n_jobs=-1
)
```

	Model	Accuracy	Precision	Recall	F1
0	Logistic Regression	0.7899	0.7487	0.6076	0.6676

=> Logistic Regression is relatively simple and interpretable, but its main limitation is that it represents the decision relationship through a relatively simple weighted structure.

- K-Nearest Neighbors

KNN follows a different idea. Instead of learning an explicit weighted function, it classifies a new observation using nearby observations from the training set.

Using Euclidean distance, the distance between two feature vectors is based on the difference between their feature values. The closest k observations are used to determine the predicted class.

$$d(x, x_i) = \sqrt{\sum_{j=1}^d (x_j - x_{ij})^2}.$$

The important assumption is that observations that are close to each other in the feature space tend to have similar target classes.

Because KNN depends on distance, feature scale is important. This is why numerical features are standardized in the main representation.

The main hyperparameter investigated in this project is k.

```

from sklearn.neighbors import KNeighborsClassifier

model_2 = Pipeline([
    (
        "preprocess",
        preprocessor
    ),
    (
        "model",
        KNeighborsClassifier(
            n_neighbors=15
        )
    )
])

model_2_scores = cross_validate(
    model_2,
    X_train,
    y_train,
    cv=cv,
    scoring=[
        "accuracy",
        "precision",
        "recall",
        "f1"
    ],
    n_jobs=-1
)

```

	Model	Accuracy	Precision	Recall	F1
0	KNN	0.8258	0.7753	0.7059	0.7381

=> KNN is simple and can model nonlinear local relationships, but it can be sensitive to the choice of k and to the representation of the data.

- Decision Tree

Decision Tree recursively divides the feature space using selected splitting criteria.

The model learns a tree structure consisting of decision rules, internal split nodes, and leaf nodes. Each split attempts to produce groups that are more homogeneous with respect to the target class.

```
from sklearn.tree import DecisionTreeClassifier

model_3 = Pipeline([
    (
        "preprocess",
        preprocessor
    ),
    (
        "model",
        DecisionTreeClassifier(
            max_depth=5,
            min_samples_leaf=8,
            random_state=RANDOM_STATE
        )
    )
])

model_3_scores = cross_validate(
    model_3,
    X_train,
    y_train,
    cv=cv,
    scoring=[
        "accuracy",
        "precision",
        "recall",
        "f1"
    ],
    n_jobs=-1
)
```

	Model	Accuracy	Precision	Recall	F1
0	Decision Tree	0.8550	0.8502	0.7197	0.7760

=> Decision Trees can model nonlinear relationships and interactions between features. They are also relatively easy to interpret.

However, a tree that becomes too complex may fit the training data too closely and generalize poorly to unseen observations.

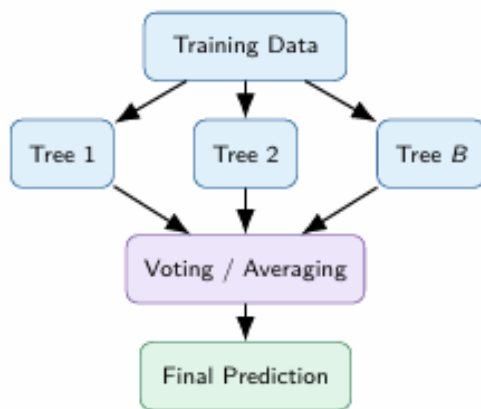
- Random Forest

Random Forest extends the Decision Tree idea by combining many trees.

Each tree is trained using a bootstrap sample, and each split considers a random subset of features. The final prediction is obtained by combining the predictions of the trees.

The basic idea can be summarized as:

Many trees → Combined prediction



```

from sklearn.ensemble import RandomForestClassifier

model_4 = Pipeline([
    (
        "preprocess",
        preprocessor
    ),
    (
        "model",
        RandomForestClassifier(
            n_estimators=200,
            max_depth=8,
            min_samples_leaf=4,
            random_state=RANDOM_STATE,
            n_jobs=-1
        )
    )
])

model_4_scores = cross_validate(
    model_4,
    X_train,
    y_train,
    cv=cv,
    scoring=[
        "accuracy",
        "precision",
        "recall",
        "f1"
    ],
    n_jobs=-1
)

```

	Model	Accuracy	Precision	Recall	F1
0	Random Forest	0.8778	0.8328	0.8131	0.8227

=> The main strength of Random Forest is that multiple diverse trees can provide a more robust prediction than a single decision tree.

However, the main disadvantages are greater computational cost and lower interpretability compared with a single tree.

Overall, these four models provide four different learning mechanisms over the same general feature-vector representation.

7. Experimental Design

The goal is not simply to run several models and report a number. A stronger experiment asks why the result occurred and investigates meaningful changes in the model, hyperparameters, or representation.

Three controlled experiments were conducted.

- Experiment 1: Model Comparison

The first experiment compares the four traditional machine learning models using the same training data, preprocessing procedure, and evaluation protocol.

The experimental question is:

+ Under the same preprocessing and the same 5-fold cross-validation protocol, which of the four traditional ML models provides the best overall classification performance?

```
for name, model in models.items():

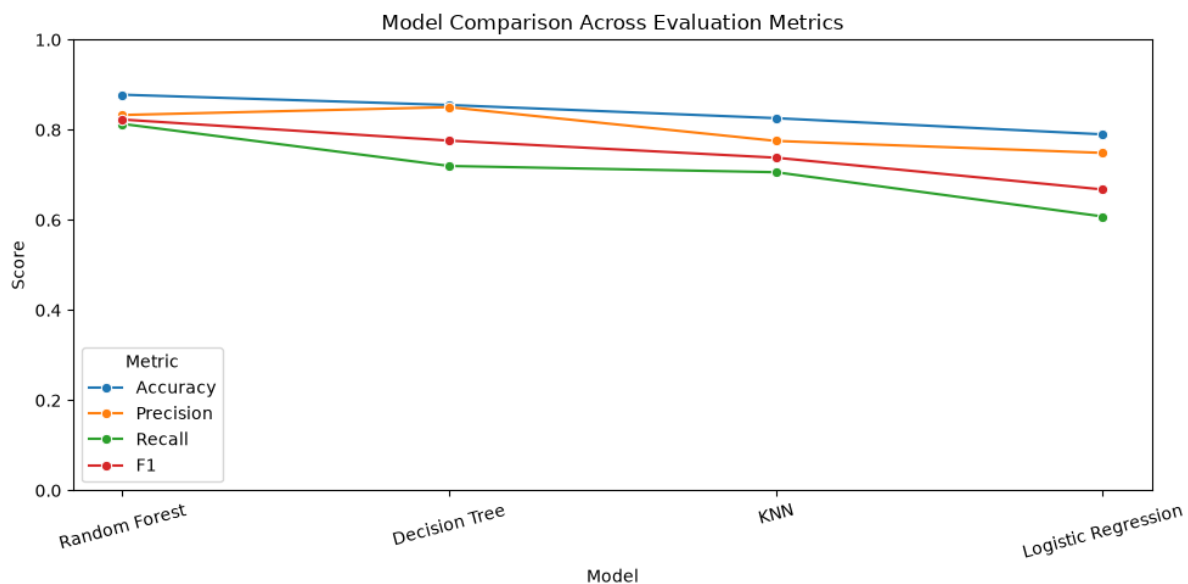
    scores = cross_validate(
        model,
        X_train,
        y_train,
        cv=cv,
        scoring=[
            "accuracy",
            "precision",
            "recall",
            "f1"
        ],
        n_jobs=-1
    )

    experiment_1_results.append({
        "Model": name,
        "Accuracy": scores["test_accuracy"].mean(),
        "Precision": scores["test_precision"].mean(),
        "Recall": scores["test_recall"].mean(),
        "F1": scores["test_f1"].mean()
    })
```

This experiment provides the main evidence used for model selection.

	Model	Accuracy	Precision	Recall	F1
0	Random Forest	0.8778	0.8328	0.8131	0.8227
1	Decision Tree	0.8550	0.8502	0.7197	0.7760
2	KNN	0.8258	0.7753	0.7059	0.7381
3	Logistic Regression	0.7899	0.7487	0.6076	0.6676

Best model according to CV F1: Random Forest



The model with the highest F1-score is preferred for the main application because the positive class is relatively rare and the application requires a balance between finding potential subscribers and avoiding excessive false positives.

The results should therefore be interpreted using multiple metrics rather than Accuracy alone.

Best model according to cross-validated F1: Decision Tree

- Experiment 2: Hyperparameter Investigation

The second experiment investigates the KNN hyperparameter k .

The experimental question is:

+ How does the KNN hyperparameter k affect classification performance?

```

k_values = [
    3, 5, 7, 9, 11,
    13, 15, 17, 19,
    21, 25, 31
]

experiment_2_results = []

for k in k_values:

    knn_model = Pipeline([
        (
            "preprocess",
            preprocessor
        ),
        (
            "model",
            KNeighborsClassifier(
                n_neighbors=k
            )
        )
    ])

    scores = cross_validate(
        knn_model,
        X_train,
        y_train,
        cv=cv,
        scoring=[
            "accuracy",
            "precision",
            "recall",
            "f1"
        ],
        n_jobs=-1
    )

```

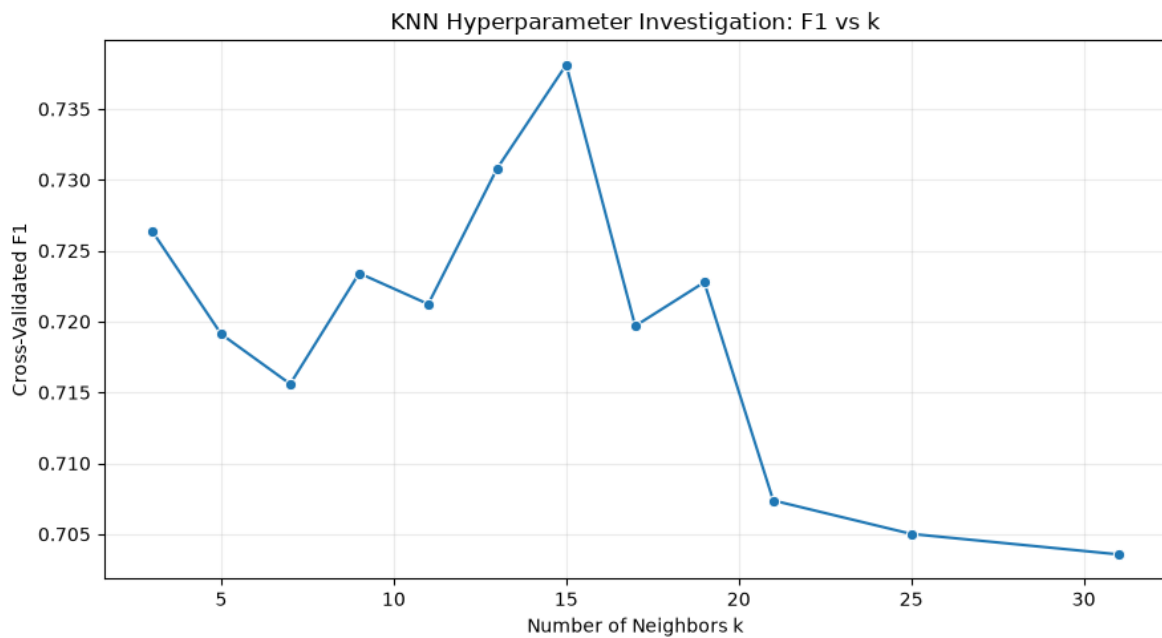
Only k is changed while the other conditions remain fixed.

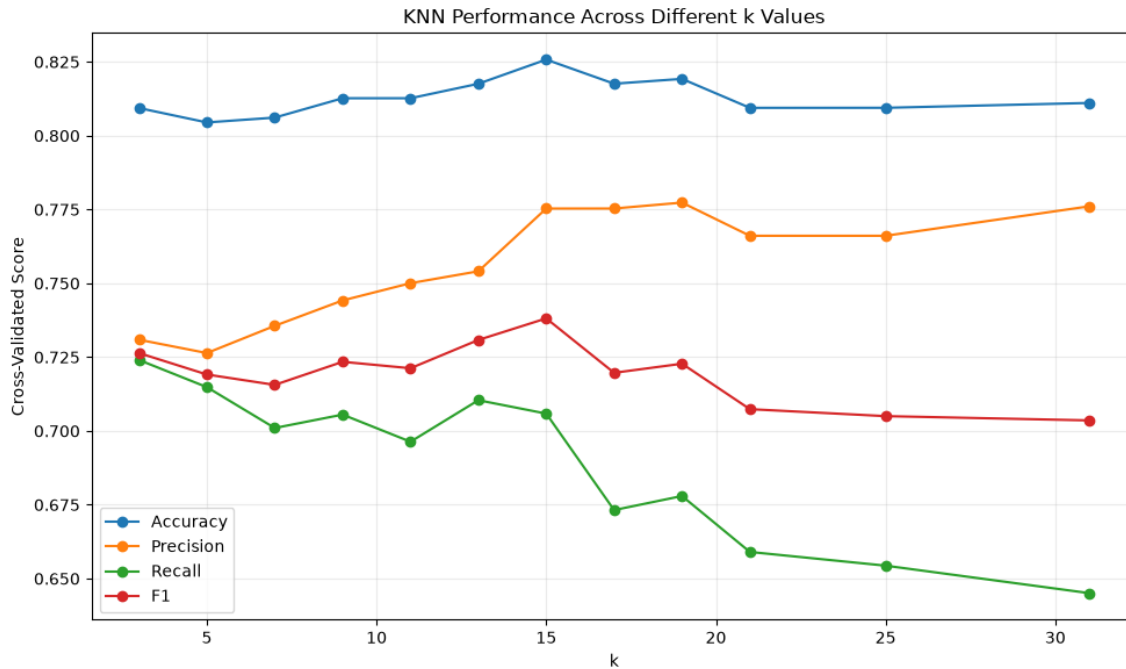
This makes the experiment controlled and allows us to observe how the neighborhood size affects performance.

	k	Accuracy	Precision	Recall	F1
0	3	0.8094	0.7309	0.7240	0.7264
1	5	0.8045	0.7264	0.7148	0.7191
2	7	0.8061	0.7356	0.7010	0.7156
3	9	0.8127	0.7442	0.7055	0.7234
4	11	0.8127	0.7500	0.6963	0.7212
5	13	0.8176	0.7541	0.7104	0.7308
6	15	0.8258	0.7753	0.7059	0.7381
7	17	0.8176	0.7753	0.6732	0.7197
8	19	0.8192	0.7773	0.6780	0.7228
9	21	0.8094	0.7661	0.6590	0.7074
10	25	0.8094	0.7661	0.6544	0.7050
11	31	0.8111	0.7761	0.6451	0.7036

Best k according to CV F1: 15

Best CV F1: 0.7381





Best k according to cross-validated F1: 15

- Experiment 3: Representation Investigation

The third experiment compares scaled and unscaled numerical features for KNN.

The experimental question is:

+ Does standardizing numerical features improve KNN performance compared with using unscaled numerical features?

```

preprocessor_scaled = ColumnTransformer(
    transformers=[
        (
            "num",
            StandardScaler(),
            numeric_features
        ),
        (
            "cat",
            encoder_scaled,
            categorical_features
        )
    ]
)

knn_scaled = Pipeline([
    ("preprocess", preprocessor_scaled),
    (
        "model",
        KNeighborsClassifier(
            n_neighbors=best_k
        )
    )
])

```

```

preprocessor_unscaled = ColumnTransformer(
    transformers=[
        (
            "num",
            "passthrough",
            numeric_features
        ),
        (
            "cat",
            encoder_unscaled,
            categorical_features
        )
    ]
)

knn_unscaled = Pipeline([
    ("preprocess", preprocessor_unscaled),
    (
        "model",
        KNeighborsClassifier(
            n_neighbors=best_k
        )
    )
])

```

```

scaled_scores = cross_validate(
    knn_scaled,
    X_train,
    y_train,
    cv=cv,
    scoring=[
        "accuracy",
        "precision",
        "recall",
        "f1"
    ],
    n_jobs=-1
)

unscaled_scores = cross_validate(
    knn_unscaled,
    X_train,
    y_train,
    cv=cv,
    scoring=[
        "accuracy",
        "precision",
        "recall",
        "f1"
    ],
    n_jobs=-1
)

```

This experiment is directly connected to the central course principle that representation is not a neutral choice. Because KNN uses distances, changing feature scale can change which observations are considered neighbors.

	Representation	Accuracy	Precision	Recall	F1
0	Scaled numerical features	0.8258	0.7753	0.7059	0.7381
1	Unscaled numerical features	0.8583	0.7977	0.7993	0.7975

8. Results

- A majority-class baseline was established before evaluating the machine learning models.

	Metric	CV Mean
0	Accuracy	0.6515
1	Precision	0.0000
2	Recall	0.0000
3	F1	0.0000

Majority class: 0

- A majority-class baseline was established before comparing the machine learning models.

The baseline always predicts the majority class.

Because approximately 65.1 percent of observations belong to Outcome 0, this simple strategy already provides a substantial Accuracy score without learning relationships between features and the target.

This demonstrates why a baseline is important. A machine learning model should provide meaningful improvement beyond a simple majority-class strategy.

- The four models were then evaluated using Accuracy, Precision, Recall, F1-score, and Confusion Matrix.

The cross-validation results obtained in the project were:

```
final_candidates = {
    "Logistic Regression": Pipeline([
        ("preprocess", preprocessor),
        (
            "model",
            LogisticRegression(
                max_iter=1000,
                random_state=RANDOM_STATE
            )
        )
    ]),
    "KNN": Pipeline([
        ("preprocess", preprocessor),
        (
            "model",
            KNeighborsClassifier(
                n_neighbors=best_k
            )
        )
    ]),
    "Decision Tree": Pipeline([
        ("preprocess", preprocessor),
        (
            "model",
            DecisionTreeClassifier(
                max_depth=8,
                min_samples_leaf=10,
                random_state=RANDOM_STATE
            )
        )
    ]),
}
```

```
    "Random Forest": Pipeline([
        ("preprocess", preprocessor),
        (
            "model",
            RandomForestClassifier(
                n_estimators=200,
                max_depth=12,
                min_samples_leaf=5,
                random_state=RANDOM_STATE,
                n_jobs=-1
            )
        )
    ])
}
```

```

final_selection_results = []

for name, model in final_candidates.items():

    scores = cross_validate(
        model,
        X_train,
        y_train,
        cv=cv,
        scoring=[
            "accuracy",
            "precision",
            "recall",
            "f1"
        ],
        n_jobs=-1
    )

    final_selection_results.append({
        "Model": name,
        "Accuracy": scores["test_accuracy"].mean(),
        "Precision": scores["test_precision"].mean(),
        "Recall": scores["test_recall"].mean(),
        "F1": scores["test_f1"].mean()
    })

```

Cross-validation results used for final model selection:

	Model	CV Accuracy	CV Precision	CV Recall	CV F1
0	Random Forest	0.8778	0.8328	0.8131	0.8227
1	Decision Tree	0.8550	0.8502	0.7197	0.7760
2	KNN	0.8258	0.7753	0.7059	0.7381
3	Logistic Regression	0.7899	0.7487	0.6076	0.6676

Selected final model: Random Forest

Selection criterion: highest CV F1 = 0.8227

These results show a clear difference between the four learning mechanisms.

Logistic Regression produced the weakest overall F1-score.

KNN improved both Recall and F1 compared with Logistic Regression.

Decision Tree produced a further improvement.

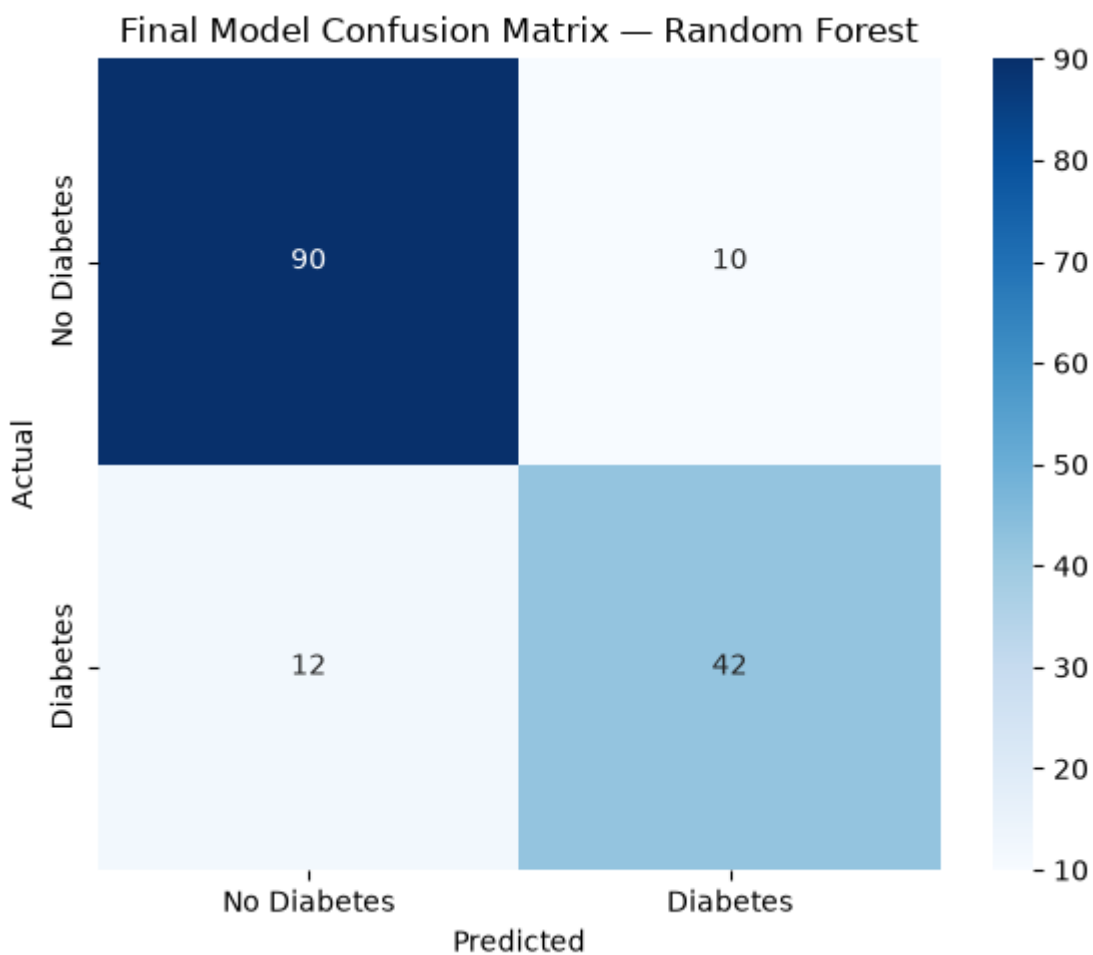
Random Forest achieved the highest values across the main evaluation criteria overall, including the highest F1-score of approximately 0.8227.

Therefore, based on the predefined F1-based selection criterion, Random Forest is the strongest candidate among the four models.

Final model trained on the complete training set.

Final evaluation on the untouched test set:

	Metric	Final Test Score
0	Accuracy	0.8571
1	Precision	0.8077
2	Recall	0.7778
3	F1	0.7925



9. Model Comparison

The model comparison demonstrates that different learning mechanisms produce different results even when the same numerical representation is used.

Logistic Regression produced the lowest overall performance among the four models. This is consistent with its relatively simple weighted relationship, which may not capture all nonlinear interactions among the medical features.

KNN performed better than Logistic Regression, indicating that local similarity in the standardized feature space contains useful information for this classification task.

Decision Tree achieved stronger results than both Logistic Regression and KNN. Its ability to represent nonlinear decision rules and interactions between features provides greater flexibility.

Random Forest achieved the strongest overall performance.

Its cross-validated results were:

Accuracy = 0.8778

Precision = 0.8328

Recall = 0.8131

F1 = 0.8227

The relatively high Recall indicates that Random Forest identifies a larger proportion of positive cases than the other evaluated models.

The combination of Precision and Recall also produces the highest F1-score.

Therefore, Random Forest is selected as the final model under the predefined F1-based model-selection criterion.

It is important to note that this conclusion is based on cross-validation results from the training set. The held-out test set is not used for selecting the model.

10. Representation Analysis

The dataset consists entirely of numerical features, so the representation process is relatively simple compared with datasets containing categorical variables.

The representation consists of eight numerical measurements that are converted into a standardized feature vector.

This representation preserves important information such as glucose concentration, BMI, age, blood pressure, insulin, and other patient measurements.

However, the representation is limited by the information contained in the dataset. It does not contain other potentially relevant medical information such as detailed medical history, medication, laboratory measurements outside the selected variables, or temporal patient information.

The representation investigation uses KNN because KNN depends directly on distances in feature space.

For the current experiment, the scaled KNN with produced:

Accuracy = 0.8258

Precision = 0.7753

Recall = 0.7059

F1 = 0.7381

The unscaled representation produced:

Accuracy = 0.8583

Precision = 0.7977

Recall = 0.7993

F1 = 0.7975

Therefore, in this particular experiment, the unscaled representation produced a higher F1-score than the standardized representation.

This is an important result because it shows that scaling should not automatically be assumed to improve performance. The effect depends on the interaction between the representation and the learning algorithm.

For KNN, changing the representation changes the distance structure of the feature space, which can change the selected neighbors and therefore the prediction.

11. Intelligent Application

The final model is integrated into a small prediction application.

The application follows the complete pipeline:

Input

→ Feature Representation

→ Preprocessing

→ Final Model

→ Prediction

→ Output

A user can enter the eight patient measurements.

The input is passed through the same preprocessing pipeline used during training.

The final Random Forest model then generates the classification prediction.

The application returns the predicted class:

0 → No diabetes predicted

1 → Diabetes predicted

The application demonstrates how a trained machine learning model can be transformed into a usable decision-support component.

The model is not treated as an independent application. Instead, the trained model is integrated with input handling, preprocessing, prediction logic, and a user interface

Diabetes Risk Intelligence Dashboard

Explore dataset signals, model performance, and run a prediction using the final model.

FINAL MODEL
Random Forest
200 trees · max_depth=8 · min_samples_leaf=4

TEST ACCURACY

85.7%

Final hold-out

TEST PRECISION

80.8%

Positive prediction quality

TEST RECALL

77.8%

Positive case coverage

TEST F1

79.2%

Primary selection metric

TEST ROC-AUC

0.943

Probability ranking

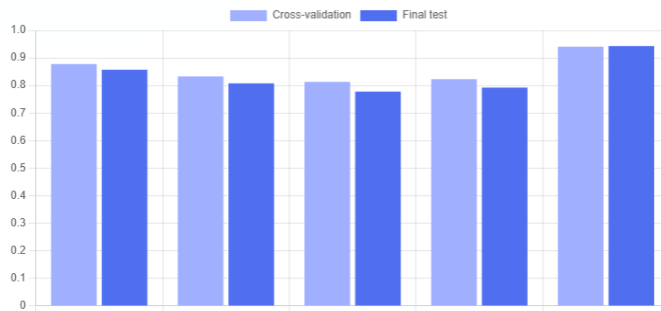
DATASET

768

8 features

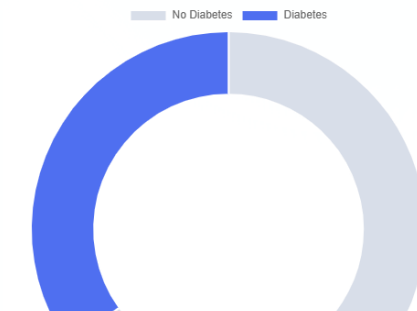
Model Performance

Cross-validation versus final hold-out test



Outcome Distribution

Diabetes class distribution



Confusion Matrix

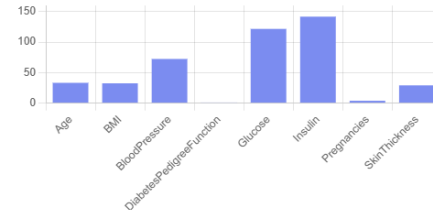
Final test-set classification behaviour

	Predicted 0	Predicted 1
Actual 0	90	10
Actual 1	12	42

TN 90 FP 10 FN 12 TP 42

Feature Overview

Average feature values in the dataset



Run a Prediction

Enter the eight measurements used by the final Random Forest model.

POST /api/predict

Pregnancies

1

Glucose

120

BloodPressure

70

SkinThickness

20

Insulin

79

BMI

25

DiabetesPedigreeFunction

0.35

Age

30

Predict with Final Model

Run a Prediction

Enter the eight measurements used by the final Random Forest model.

POST /api/predict

Pregnancies	Glucose	BloodPressure	SkinThickness
1	120	70	20
Insulin	BMI	DiabetesPedigreeFunction	Age
79	25	0.35	30

Predict with Final Model

NEGATIVE RESULT

No diabetes predicted

5.5%
positive probability

No diabetes 94.5%

Diabetes 5.5%

Structured Data → Feature Vector → Traditional ML → Application

Test set reserved for final evaluation

12. Limitations

The developed system has several limitations.

First, the model depends on the quality and representativeness of the dataset. If future patients differ substantially from the observations used during training, the model's performance may decrease.

Second, the feature representation contains only eight numerical measurements. Information that is not included in these features cannot be learned by the model.

Third, some numerical values may contain zero entries that could represent unavailable measurements rather than meaningful physiological values. This introduces a representation issue that may affect model performance.

Fourth, the system performs classification rather than medical diagnosis. A prediction from the model should not be interpreted as a definitive clinical decision.

Finally, the system does not reason about medical context, maintain long-term patient state, or adapt continuously to new clinical evidence.

The system should therefore be viewed as a small predictive intelligent system and not as a complete medical reasoning system.

13. Reflection

The main lesson from this project is that developing a machine learning system involves much more than selecting an algorithm and training it.

The first important lesson is representation. The model receives a numerical representation of the patient rather than the real-world patient itself. Consequently, the information that can be learned is limited by the selected features.

The second lesson is that different learning mechanisms behave differently on the same representation. Logistic Regression, KNN, Decision Tree, and Random Forest produced noticeably different performance despite receiving the same underlying input features.

The third lesson is the importance of controlled experiments. The hyperparameter experiment showed how the choice of affects KNN, while the representation experiment showed that changing feature scaling can also change model performance.

The fourth lesson is that evaluation must be connected to the application objective. Accuracy alone does not provide enough information about classification performance, particularly when the classes are not perfectly balanced. Precision, Recall, and F1 provide additional information about positive-class behavior.

The fifth lesson is the importance of keeping the test set separate from model development. Cross-validation on the training set was used for model comparison and experiment design, while the held-out test set was reserved for final evaluation after model selection.

The project therefore demonstrates that intelligent-system development is a complete process involving problem understanding, representation, learning, experimentation, evaluation, and application.

14. Conclusion

This project developed a small intelligent system for diabetes classification using structured medical data and traditional machine learning.

The raw patient measurements were transformed into a numerical feature-vector representation through median imputation and standardization.

A majority-class baseline was established before training the main models.

Four traditional machine learning algorithms were compared:

Logistic Regression

K-Nearest Neighbors

Decision Tree

Random Forest

Three controlled experiments were conducted.

The first experiment compared the four learning mechanisms.

The second investigated the effect of the KNN hyperparameter , with producing the highest cross-validated F1-score among the investigated values.

The third investigated the effect of numerical feature scaling on KNN and showed that the unscaled representation produced a higher F1-score under the tested conditions.

The cross-validation model comparison showed that Random Forest achieved the strongest overall performance, with an F1-score of approximately 0.8227.

Random Forest was therefore selected as the final model based on the predefined F1-based model-selection criterion.

The final model was then integrated into an application that accepts patient measurements and produces a diabetes classification prediction.

The project demonstrates the first stage of intelligent system development:

Structured Data

→ Feature Vector

→ Traditional ML

→ Evaluation

→ Application

The central lesson is that intelligent-system development is not only about training a model. It involves deciding what information to represent, selecting an appropriate learning mechanism, designing controlled experiments, evaluating evidence, and integrating the learned model into a usable system.

